



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

Отчет по лабораторной работе № 2

«Обработка пропусков в данных, кодирование категориальных признаков, масштабирование данных.»
по дисциплине «Технологии машинного обучения»

Студент ИУ5-61Б
(Группа)

Е.И. Бирюкова
(Подпись, дата) (И.О.Фамилия)

Преподаватель

А.Н. Нардид
(Подпись, дата) (И.О.Фамилия)

Москва

2025

Лабораторная работа №2

Обработка пропусков в данных, кодирование категориальных признаков, масштабирование данных.

Введение

В качестве набора данных будем использовать датасет "Wine reviews". В датасете содержится информация о сортах вин.

Ссылка на датасет: <https://www.kaggle.com/datasets/zynicide/wine-reviews>

Датасет содержит следующие поля.

1. **id** - номер по списку
2. **country** - страна производства
3. **description** - несколько фраз от сомелье, описывающих вкус, запах, внешний вид, ощущения от вина и так далее
4. **designation** - виноградник на территории винодельни, откуда собирают виноград, из которого делают вино
5. **points** - количество баллов, на которые любители вина оценили вино по шкале от 1 до 100 (хотя они говорят, что публикуют отзывы только винах, которые набрали >80 баллов)
6. **price** - стоимость бутылки вина
7. **province** - провинция или штат, из которого произведено вино
8. **region_1** - винодельческий регион в провинции или штате (например, Напа)
9. **region_2** - иногда в пределах винодельческого региона указываются более конкретные регионы (например, Резерфорд в долине Напа), но иногда это значение может быть пустым
10. **variety** - сорт винограда, используемый для производства вина (например, Пино Нуар)
11. **winery** - винодельня, на которой производилось это вино

Для выбранного датасета (датасетов) на основе материалов лекции решить следующие задачи:

- обработку пропусков в данных;
- кодирование категориальных признаков;
- масштабирование данных.

Импорт библиотек

Импортируем библиотеки с помощью команды import.

```
%matplotlib notebook
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
import statsmodels.stats.multitest
```

```
from scipy import stats
```

Загрузка и очистка данных

Загрузим файлы датасета в помощью библиотеки Pandas. Проверим данные на предмет присутствия пропусков, выбросов и пр. Также немного преобразуем данные для более эффективного хранения памяти и удобного использования в будущем.

```
df = pd.read_csv('wine_views.csv')
df.head(10)
```

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
0	0	US	This tremendous 100% varietal	Martha's Vineyard	96	235.0	California	Napa Valley	Napa	Cabernet Sauvignon	Heitz

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
			wine hails from ...								
1	1	Spain	Ripe aromas of fig, blackberry and cassis are ...	Carodorum Selección Especial Reserva	96	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodrígu
2	2	US	Mac Watson honors the memory of a wine once ma...	Special Selected Late Harvest	96	90.0	California	Knights Valley	Sonoma	Sauvignon Blanc	Macaul
3	3	US	This spent 20 months in 30% new French oak, an...	Reserve	96	65.0	Oregon	Willamette Valley	Willamette Valley	Pinot Noir	Ponzi
4	4	France	This is the top wine from La Bégude, named aft...	La Brûlade	95	66.0	Provence	Bandol	NaN	Provence red blend	Domainir de la Bégude
5	5	Spain	Deep, dense and pure from the opening bell, th...	Numanthia	95	73.0	Northern Spain	Toro	NaN	Tinta de Toro	Numan
6	6	Spain	Slightly gritty black-fruit aromas include a s...	San Román	95	65.0	Northern Spain	Toro	NaN	Tinta de Toro	Mauroc
7	7	Spain	Lush cedary black-fruit aromas are luxe and of...	Carodorum Único Crianza	95	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodrígu
8	8	US	This re-named vineyard was formerly bottled as...	Silice	95	65.0	Oregon	Chehalem Mountains	Willamette Valley	Pinot Noir	Bergstr
9	9	US	The producer sources from two blocks of the vi...	Gap's Crown Vineyard	95	60.0	California	Sonoma Coast	Sonoma	Pinot Noir	Blue Fa

```
# Проверка на пропуски
print("Пропуски до обработки:")
print(df.isna().sum())
```

```
Пропуски до обработки:
Unnamed: 0      0
country        5
description     0
designation    45735
points         0
```

```
price          13695
province        5
region_1       25060
region_2       89977
variety         0
winery          0
dtype: int64
```

```
# Размер датасета - 150930 строк, 11 колонок
df.shape
```

```
(150930, 11)
```

```
total_count = df.shape[0]
print('Всего строк: {}'.format(total_count))
# Список колонок
# df.columns
# Список колонок с типами данных
df.dtypes
```

```
Всего строк: 150930
```

```
Unnamed: 0      int64
country         object
description      object
designation      object
points          int64
price          float64
province        object
region_1        object
region_2        object
variety         object
winery          object
dtype: object
```

```
# Основные статистики для категориальных полей
print(df.select_dtypes('object').describe())
```

	country	description	designation \
count	150925	150930	105195
unique	48	97821	30621
top	US	A little bit funky and unsettled when you pop ...	Reserve
freq	62397	6	2752

	province	region_1	region_2	variety	winery
count	150925	125870	60953	150930	150930
unique	455	1236	18	632	14810
top	California	Napa Valley	Central Coast	Chardonnay	Williams Selyem
freq	44508	6209	13057	14482	374

```
# Основные статистики для числовых полей
print(df.describe())
```

	Unnamed: 0	points	price
count	150930.000000	150930.000000	137235.000000
mean	75464.500000	87.888418	33.131482
std	43569.882402	3.222392	36.322536
min	0.000000	80.000000	4.000000
25%	37732.250000	86.000000	16.000000
50%	75464.500000	88.000000	24.000000
75%	113196.750000	90.000000	40.000000
max	150929.000000	100.000000	2300.000000

```
# Определим уникальные значения для категориальных полей
fields = ['country', 'province', 'region_1', 'region_2', 'variety']

# Используем цикл для итерации по полям и вывода уникальных значений
for field in fields:
    print(f"Уникальные значения для поля '{field}':")
    print(df[field].unique())
    print("-" * 30) # Просто для визуального разделения результатов
```

```
Уникальные значения для поля 'country':
['US' 'Spain' 'France' 'Italy' 'New Zealand' 'Bulgaria' 'Argentina'
 'Australia' 'Portugal' 'Israel' 'South Africa' 'Greece' 'Chile' 'Morocco'
 'Romania' 'Germany' 'Canada' 'Moldova' 'Hungary' 'Austria' 'Croatia'
 'Slovenia' nan 'India' 'Turkey' 'Macedonia' 'Lebanon' 'Serbia' 'Uruguay'
 'Switzerland' 'Albania' 'Bosnia and Herzegovina' 'Brazil' 'Cyprus'
 'Lithuania' 'Japan' 'China' 'South Korea' 'Ukraine' 'England' 'Mexico'
 'Georgia' 'Montenegro' 'Luxembourg' 'Slovakia' 'Czech Republic' 'Egypt'
 'Tunisia' 'US-France']
-----

Уникальные значения для поля 'province':
['California' 'Northern Spain' 'Oregon' 'Provence' 'Northeastern Italy'
 'Southwest France' 'Kumeu' 'Washington' 'Bulgaria' 'Tuscany'
 'France Other' 'Rhône Valley' 'Galicia' 'Andalucia' 'Idaho' 'Burgundy'
 'Loire Valley' 'New York' 'Mendoza Province' 'Victoria' 'Alentejano'
 'Piedmont' 'Alentejo' 'Champagne' 'Upper Galilee' 'Beira Atlantico'
 'Veneto' 'Douro' 'Tejo' 'Stellenbosch' 'Levante' 'Sicily & Sardinia'
 'Southern Italy' 'Languedoc-Roussillon' 'Bordeaux' 'Atalanti Valley'
 'Catalonia' 'Santorini' 'Florina' 'Marchigue' 'Colchagua Valley'
 'Curicó Valley' 'Nemea' 'Maule Valley' 'Alsace' 'Guerrouane'
 'Colinele Dobrogei' 'Central Spain' 'Vinho Verde' 'Mosel' 'Rheinhessen'
 'Golan Heights' 'Württemberg' 'Ahr' 'British Columbia' 'Moldova'
 'Spain Other' 'Sopron' 'Other' 'Walker Bay' 'Dão' 'Italy Other'
 'Duriense' 'Ontario' 'Beiras' 'Tokaji' 'Lisboa' 'Thermenregion'
 'Burgenland' 'Carnuntum' 'Rheingau' 'Nahe' 'South Australia'
 'North Dalmatia' 'Thracian Valley' 'Goriska Brda' 'Western Cape'
 'Overberg' 'Robertson' 'Galilee' 'Maipo Valley' 'Casablanca Valley'
 'Cachapoal Valley' 'Terras do Dão' 'Leyda Valley' 'Peumo' 'Baden'
 'Limarí Valley' 'Lombardy' 'Peloponnese' 'Judean Hills' 'Tasmania'
 'Bairrada' 'Simonsberg-Paarl' 'Portuguese Table Wine' 'Beaujolais'
 'Península de Setúbal' 'Aconcagua Valley' 'Virginia'
 'Dealurile Munteniei' nan 'Nashik' 'Niederösterreich' 'Marlborough'
 'Central Otago' 'Palmela' 'Beira Interior' 'Port' 'Franken'
 'Martinborough' 'Wagram' 'Kamptal' 'South Africa' 'Rapel Valley' 'Pfalz'
 'Hvar' 'Colorado' 'Arizona' 'Aconcagua Costa' 'Zenata' 'Nelson'
 'Elqui Valley' 'San Antonio' 'Eisenberg' 'Istria' 'Ankara' 'Villány'
 'Štajerska' 'Moscatel de Setúbal' 'Moscatel do Douro' 'Samson' 'Wachau'
 'Tikves' 'Lebanon' 'Kremstal' 'Pocerina' 'Župa' 'Vienna' 'Morocco'
 "Hawke's Bay" 'Juanico' 'Montevideo' 'Kras' 'Dolenjska' 'Korčula'
 'America' 'Bío Bío Valley' 'Weinland Österreich' 'Weinviertel'
 'Wiener Gemischter Satz' 'Bekaa Valley' 'Aegean' 'Central Italy' 'Galil'
 'Recas' 'Turkey' 'Bucelas' 'Chalkidiki' 'Australia Other'
```

'Jerusalem Hills' 'Ilia' 'Danube River Plains' 'Valais' 'Neuchâtel'
'North Carolina' 'Mirditë' 'Corinth' 'Waipara' 'Vipavska Dolina'
'Western Australia' 'Macedonia' 'Minho' 'Viile Timisului' 'Pangeon'
'Romania' 'Central Greece' 'Central Valley' 'Awatere Valley' 'Alenteo'
'Thrace' 'Chile' 'New South Wales' 'Primorska' 'Slovenia' 'Progreso'
'Retsina' 'Apalta' 'San Clemente' 'Dealu Mare' 'Alenquer' 'Canelones'
'New Zealand' 'Mostar' 'Shomron' 'Haut-Judeé' 'Drama' 'Slovenska Istra'
'Pannon' 'Spanish Islands' 'Fruška Gora' 'Gisborne' 'Terasale Dunarii'
'Uruguay' 'Serra Gaúcha' 'Südsteiermark' 'Lontué Valley' 'New Mexico'
'Lolol Valley' 'Missouri' 'Wagram-Donau' 'Curicó and Leyda Valleys'
'Loncomilla Valley' 'Mittelburgenland' 'Itata Valley' 'Swartland'
'Leithaberg' 'Casablanca & Leyda Valleys' 'Pirque' 'Hemel en Aarde'
'Waitaki Valley' 'Simonsberg-Stellenbosch' 'Mantinia' 'Tulbagh'
'Canterbury' 'Choapa Valley' 'Negev' 'Paarl' 'Setubal' 'Algarve'
'Coastal Region' 'Commandaria' 'Patras' 'Jonkershoek Valley' 'Elgin'
'Kathikas' 'Pennsylvania' 'Dan' 'Vin de Pays de Velvendo' 'Ribatejo'
'Durbanville' 'Polkadraai Hills' 'Mosel-Saar-Ruwer'
'Österreichischer Sektor' 'Crete' 'Südoststeiermark' 'Malgas' 'Styria'
'Donau' 'Omaka Valley' 'Rapsani' 'Opuntia Locris' 'Samos'
'Vin de Pays de Macédoine' 'Greece' 'Halkidiki' 'Attica' 'Cephalonia'
'Mount Athos' 'Vinho Espumante de Qualidade' 'South Island'
'Neusiedlersee-Hügelland' 'Portugal' 'Traisental' 'Austria' 'Bot River'
'Cederberg' 'Wellington' 'Neusiedlersee' 'Ohio' 'Epanomi' 'Korinthia'
'Hungary' 'Neszmély' 'Lithuania' 'Breedekloof' 'Kyoto' 'Franschhoek'
'Naoussa' 'Thraci' 'Constantia' 'China' 'Jiri Valley' 'Cape Agulhas'
'Massachusetts' 'Arcadia' 'New Jersey' 'Pageon' 'Northwestern Italy'
'Puente Alto' 'Klein River' 'Bui' 'Mavrodaphne of Patras' 'Ilias'
'Távora-Varosa' 'Requinoa' 'Colares' 'Peljesac' 'Elazığ' 'Sebes'
'Ukraine' 'Vinho da Mesa' 'Wairau Valley' 'Terras do Sado' 'Israel'
'Michigan' 'Texas' 'England' 'Valle de Guadalupe' 'Tokaj' 'Imathia'
'Malleco' 'Kakheti' 'Brda' 'Waiheke Island' 'Waipara Valley' 'Amyndeon'
'Sterea Ellada' 'Transmontanas' 'Dingač' 'Vinho Espumante' 'Croatia'
'Lemnos' 'Krk' 'Terras da Beira' 'Valle de Guadalupe and Santo Tomás'
'Wairarapa' 'Georgia' 'Rapel Valley-Casablanca Valley'
'Lake Skadar Valley' 'Middle and South Dalmatia' 'Steiermark'
'Goumenissa' 'Estremadura' 'Obidos' 'Washington-Oregon' 'Vinho Licoroso'
'Mátra' 'Achaia' 'Cyclades' 'Trás-os-Montes' 'Darling'
'Vale dos Vinhedos' 'Tradouw' 'San Jose' 'Brazil' 'Iowa' 'Germany'
'Santa Cruz' 'Philadelphia' 'Torres Vedras' 'Molina'
'Casablanca-Curicó Valley' 'Kutjevo' 'Madeira' 'East Coast' 'Messinia'
'Ribatejano' 'Negev Hills' 'Groenekloof' 'Thessalikos' 'Cyprus'
'Eilandia' 'Dalmatian Coast' 'Muscat of Patras' 'Pitsilia Mountains'
'Queensland' 'Danube Hills Valley' 'Eger' 'Rhein-Mosel' 'Sliven'
'Targovishte' 'Tarnave' 'Peza' 'Szekszárd' 'Nápoly' 'Nápoly'
'Plaiurile Drancei' 'Transylvanian Plateau' 'Felso-Magyarország'
'North Island' 'Moselle Luxembourgeoise' 'Chisinau County'
'Murfatlar Cernavoda' 'Eastern Bays' 'Marlborough & Hawkes Bay'
'Ohiti Valley' 'Moutere' 'El Colorado' 'Colonia' 'Atlantida' 'Vânju Mare'
'Muzla' 'Murfatlar' 'Dealurile Huşilor' 'Moravia' 'Transmontano'
'Black Sea Coastal' 'Curicó and Maipo Valleys' 'Cape South Coast'
'Cauquenes Valley' 'Letrinon' 'Santa Catarina' 'Serra do Sudeste'
'Vale Trentino' 'Podunavlje' 'Krania Olympus' 'Pafos'
'Central Otago-Marlborough' 'Lesbos' 'Evia' 'Monemvasia' 'Northern Cape'
'Ismarikos' 'Vermont' 'Connecticut' 'San Antonio de las Minas Valley'
'San Vicente' 'Rio Claro' 'Maipo Valley-Colchagua Valley'
'Colchagua Costa' 'Kentucky' 'Viile Carasului' 'Breede River Valley'
'Canada Other' 'Zitsa' 'Lakonia' 'Beotia' 'Table wine' 'Cahul' 'Pisatis'
'Langenlois' 'Amindeo' 'Aigialias Slopes' 'Hrvatsko Primorje' 'Gerania'
'Egialias Slopes' 'Vin de Pays d'Imathia' 'Codru Region' 'Lemesos'
'Lutzville Valley' 'Ioannina' 'Viile Timis' 'Egypt' 'Limnos'
'Vino da Tavola della Svizzera Italiana' 'Nevada' 'Rose Valley' 'Ticino'
'Ella Valley' 'Douro Superior' 'Tunisia' 'South Styria' 'Elim'

'Martinborough Terrace' 'Santa Barbara County-Condrieu' "Beni M'Tir"
'Stirling']

Уникальные значения для поля 'region_1':

['Napa Valley' 'Toro' 'Knights Valley' ... 'Listrac' 'St.-Amour'
'Coteaux du Tricastin']

Уникальные значения для поля 'region_2':

['Napa' nan 'Sonoma' 'Willamette Valley' 'Central Coast' 'Columbia Valley'
'Finger Lakes' 'California Other' 'Mendocino/Lake Counties' 'South Coast'
'Central Valley' 'New York Other' 'Sierra Foothills' 'Southern Oregon'
'Oregon Other' 'Napa-Sonoma' 'North Coast' 'Washington Other'
'Long Island']

Уникальные значения для поля 'variety':

['Cabernet Sauvignon' 'Tinta de Toro' 'Sauvignon Blanc' 'Pinot Noir'
'Provence red blend' 'Friulano' 'Tannat' 'Chardonnay' 'Tempranillo'
'Malbec' 'Rosé' 'Tempranillo Blend' 'Syrah' 'Mavrud' 'Sangiovese'
'Sparkling Blend' 'Rhône-style White Blend' 'Red Blend' 'Mencia'
'Palomino' 'Petite Sirah' 'Riesling' 'Cabernet Sauvignon-Syrah'
'Portuguese Red' 'Nebbiolo' 'Pinot Gris' 'Meritage' 'Baga' 'Glera'
'Malbec-Merlot' 'Merlot-Malbec' 'Ugni Blanc-Colombard' 'Viognier'
'Cabernet Sauvignon-Cabernet Franc' 'Moscato' 'Pinot Grigio'
'Cabernet Franc' 'White Blend' 'Monastrell' 'Gamay' 'Zinfandel' 'Greco'
'Barbera' 'Grenache' 'Rhône-style Red Blend' 'Albariño' 'Malvasia Bianca'
'Assyrtiko' 'Malagouzia' 'Carmenère' 'Bordeaux-style Red Blend'
'Touriga Nacional' 'Agiorgitiko' 'Picpoul' 'Godello' 'Gewürztraminer'
'Merlot' 'Syrah-Grenache' 'G-S-M' 'Mourvèdre'
'Bordeaux-style White Blend' 'Petit Verdot' 'Muscat'
'Chenin Blanc-Chardonnay' 'Cabernet Sauvignon-Merlot' 'Pinot Bianco'
'Alvarinho' 'Portuguese White' 'Garganega' 'Sauvignon'
'Gros and Petit Manseng' 'Tannat-Cabernet' 'Alicante Bouschet' 'Aragonês'
'Silvaner' 'Ugni Blanc' 'Grüner Veltliner' 'Frappato' 'Lemberger'
'Sylvaner' 'Chasselas' 'Alsace white blend' 'Früburgunder' 'Kekfrankos'
'Vermentino' 'Sherry' 'Aglanico' 'Torrontés' 'Primitivo'
'Semillon-Sauvignon Blanc' 'Portuguese Rosé' 'Grenache-Syrah'
'Prié Blanc' 'Negrette' 'Furmint' 'Carignane' 'Pinot Blanc'
'Nero d'Avola' 'St. Laurent' 'Blauburgunder' 'Blaufränkisch' 'Scheurebe'
'Ribolla Gialla' 'Charbono' 'Malbec-Cabernet Sauvignon'
'Pinot Noir-Gamay' 'Pinot Nero' 'Gros Manseng' 'Nerello Mascalese'
'Shiraz' 'Negroamaro' 'Champagne Blend' 'Romorantin'
'Syrah-Cabernet Sauvignon' 'Tannat-Merlot' 'Duras' 'Garnacha'
'Tinta Francisca' 'Portuguese Sparkling' 'Chenin Blanc' 'Turbiana'
'Petite Verdot' 'Posip' 'Fumé Blanc' 'Roussanne' 'Grillo'
'Müller-Thurgau' 'Pinot Auxerrois' 'Port' 'Cabernet Blend'
'Cabernet Franc-Cabernet Sauvignon' 'Castelão' 'Encruzado'
'Touriga Nacional-Cabernet Sauvignon' 'Colombard-Sauvignon Blanc'
'Moscatel' 'Marsanne' 'Siria' 'Garnacha Blanca'
'Merlot-Cabernet Sauvignon' 'Arinto' 'Petit Manseng' 'Loureiro' 'Melon'
'Carricante' 'Fiano' 'Schwartzriesling' 'Sangiovese-Syrah'
'Tannat-Cabernet Franc' 'Cabernet Franc-Merlot'
'Sauvignon Blanc-Semillon' 'Macabeo' 'Alfrocheiro' 'Aligoté' 'Verdejo'
'Grenache Blanc' 'Fernão Pires' 'Spätburgunder' 'Ciliegiolo'
'Cabernet Sauvignon-Carmenère' 'Auxerrois' 'Sirica' 'Zweigelt'
'Pugnitello' 'Rosado' 'Rosato' 'Malvazija' 'Kalecik Karasi'
'Muskat Ottonel' 'Malbec-Bonarda' 'Tempranillo-Cabernet Sauvignon'
'Rivaner' 'Trepát' 'Baco Noir' 'Trebiano' 'Chardonnay-Viognier'
'Syrah-Mourvèdre' 'Graciano' 'Roviello' 'Perricone' 'Falanghina' 'Vranec'
'Carignan' 'Cabernet-Shiraz' 'Verdelho' 'Pedro Ximénez'
'Marsanne-Roussanne' 'Malbec Blend' 'Weissburgunder' 'Morava' 'Ruen'
'Hondarrabi Zuri' 'Catarratto' 'Chardonnay-Sauvignon Blanc' 'Vidal'
'Rieslaner' 'Dornfelder' 'Tinto Fino' 'Gelber Muskateller']

'Roter Veltliner' 'Aragonez' 'Vitovska' 'Pinot Noir-Syrah' 'Gamay Noir'
'Grauburgunder' 'Cannonau' 'Mauzac' 'Austrian Red Blend' 'Sémillon'
'Lambrusco di Sorbara' 'Teran' 'Dolcetto' 'Cinsault' 'Assyrtico'
'Teroldego' 'Tamjanika' 'Boğazkere' 'Kadarka' 'Narince'
'Malbec-Petit Verdot' 'Veltliner' 'Traminer' 'Lambrusco' 'Arneis'
'Cabernet Sauvignon-Shiraz' 'Tocai Friulano' 'Fer Servadou' 'Muskateller'
'Nerello Cappuccio' 'Moscatel Roxo' 'Elbling' 'Saperavi' 'Antão Vaz'
'Pinot Meunier' 'Petite Syrah' 'Malvasia' 'Malbec-Tannat' 'Kallmet'
'Syrah-Merlot' 'Montepulciano' 'Kerner' 'Alvarinho-Chardonnay' 'Žilavka'
'Vinhão' 'Chardonnay-Semillon' 'Carmenère-Cabernet Sauvignon'
'Merlot-Cabernet Franc' 'Orangetraube' 'Cabernet Sauvignon-Sangiovese'
'Okuzgozu' 'Viura' 'Garnacha-Syrah' 'Zibibbo' 'Feteasca' 'Xarel-lo'
'Prokupac' 'Códega do Larinho' 'Touriga Nacional Blend' 'Inzolia'
'Cabernet-Syrah' 'Lambrusco Grasparossa' 'Malagousia'
'Cabernet Franc-Malbec' 'Feteasca Neagra' 'Yapincak' 'Tempranillo-Shiraz'
'Cabernet Sauvignon Grenache' 'Tinta Roriz' 'Merlot-Syrah' 'Tinta Fina'
'Colombard-Ugni Blanc' 'Colombard' 'Roditis' 'Grenache-Carignan' 'Emir'
'Orange Muscat' 'Karalahna' 'Trincadeira' 'Refosco' 'Pied de Perdrix'
'Vignoles' 'Carignan-Grenache' "Muscat d'Alexandrie" 'Bobal' 'Symphony'
'Norton' 'Sauvignon Blanc-Sauvignon Gris' 'Rkatsiteli'
'Roussanne-Viognier' 'Pinela' 'Blatina' 'Shiraz-Viognier' 'Bonarda'
'Sauvignon Blanc-Chardonnay' 'Chambourcin' 'Traminette' 'Grenache Blend'
'Jaen' 'Mondeuse' 'Fetească Regală' 'Teroldego Rotaliano'
'Sangiovese-Cabernet Sauvignon' 'Listán Negro' 'Syrah-Petite Sirah'
'Viognier-Chardonnay' 'Kuntra' 'Jacquère' 'Portuguiser' 'Grecanico'
'Verdejo-Viura' 'Tinto del Pais' 'Moscato Giallo'
'Cabernet Sauvignon-Malbec' 'Mission' 'Neuburger' 'Bastardo' 'Bical'
'Sacy' 'Carineña' 'Garnacha-Tempranillo' 'Pecorino' 'Garnacha Blend'
'Cococciola' 'Passerina' 'Gaglioppo' 'Garnacha Tintorera' 'Prieto Picudo'
'Tempranillo Blanco' "Cesanese d'Affile" 'Muscat Canelli' 'Cabernet'
'Malvasia Nera' 'Premsal' 'Mansois' 'Welschriesling' 'Shiraz-Tempranillo'
'Verdicchio' 'Sagrantino' 'Rolle' 'Trousseau Gris' 'Counoise' 'Mantonico'
'Cariñena-Garnacha' 'Insolia' 'Tokaji' 'Austrian white blend'
'Shiraz-Grenache' 'Claret' 'Syrah-Tempranillo' 'Uva di Troia' 'Aleatico'
'Piedirosso' 'Viognier-Marsanne' 'Pinot Grigio-Sauvignon Blanc'
'Pallagrello Nero' 'Chardonnay-Albariño' 'Savagnin' 'Pinotage' 'Braucol'
'Moschofilero' 'Nero di Troia' 'Carignano' 'Susumaniello'
'Baga-Touriga Nacional' 'Vidal Blanc' 'Vernaccia'
'Corvina, Rondinella, Molinara' 'Mavrotragano' 'Garnacha-Monastrell'
'Lagrein' 'Cabernet Merlot' 'Monastrell-Syrah' 'Malbec-Tempranillo'
'Syrah-Viognier' 'Verdeca' 'Sangiovese Grosso' 'Merlot-Argaman'
'Chenin Blanc-Viognier' 'Garnacha-Cabernet' 'Maturana' 'Malvar' 'Airen'
'Monica' 'Gewürztraminer-Riesling' 'Prugnolo Gentile' 'Steen'
'Chenin Blanc-Sauvignon Blanc' 'Shiraz-Cabernet Sauvignon' 'Picolit'
'Prosecco' 'White Riesling' 'White Port' 'Zierfandler' 'Petroutianos'
'Mavrodaphne' 'Savatiano' 'Tempranillo-Garnacha' 'Vidadillo'
'Syrah-Cabernet' 'Gelber Traminer' 'Grenache-Shiraz' 'Rotgipfler'
'Cabernet Sauvignon-Tempranillo' 'Edelzwicker' 'Cortese'
'Chardonnay Weissburgunder' 'Torbato' 'Verduzzo' 'Debit' 'Bovale'
'Tempranillo-Merlot' 'Xinisteri' 'Merlot-Cabernet'
'Verdejo-Sauvignon Blanc' 'Black Muscat' 'Koshu' 'Királyleányka'
'Favorita' 'Xinomavro' 'Cserszegi Fűszeres' 'Hárslevelű' 'Pallagrello'
'Mavroudi' 'Muscat Blanc' 'Schiava' 'Meoru' 'Nuragus'
'Trebiano di Lugana' 'Coda di Volpe' 'Raboso' 'Shiraz-Pinotage'
'Enantio' 'Greco Bianco' 'Tai' 'Tokay' 'Muscadel'
'Cabernet Franc-Carmenère' 'Tintilia' 'Segalin' 'Lacrima' 'Cerceal'
'Cayuga' 'Sauvignon Gris' 'Albana' 'Corvina' 'Macabeo-Moscatel'
'Macabeo-Chardonnay' 'Moscadello' 'Nasco' 'Viognier-Roussanne'
'Plavac Mali' 'Cabernet Sauvignon-Merlot-Shiraz'
'Sauvignon Blanc-Chenin Blanc' 'Shiraz-Mourvèdre' 'Albarín'
'Black Monukka' 'Morio Muskat' 'Nielluciu' 'Alicante'
'Cabernet Sauvignon and Tinta Roriz' 'Viura-Chardonnay' "Loin de l'Oeil"

'Roter Traminer' 'Karasakiz' 'Casavecchia' 'Malvasia-Viura' 'Nosiola'
 'Incrocio Manzoni' 'Viura-Verdejo' 'Erbaluce' 'Forcallà' 'Pansa Blanca'
 'Catalanesca' 'Muscadelle' 'Malbec-Syrah' 'Petit Meslier'
 'Johannisberg Riesling' 'Pignoletto' 'Cabernet Pfeffer'
 'Syrah-Cabernet Franc' 'Valdiguié' 'Mazuelo' 'Brachetto' 'Jacquez'
 'Moscofilero' 'Chardonnay-Sauvignon' 'Madeleine Angevine' 'Ruché'
 'Merlot-Petite Verdot' 'Roussanne-Marsanne' 'Moscatel de Alejandría'
 'Muscat Blanc à Petit Grain' 'Sämling' 'Mtsvane' 'Zlahtina' 'Zelen'
 'Doña Blanca' 'Carmenère-Syrah' 'Roussanne-Grenache Blanc'
 'Kinali Yapincak' 'Robola' 'Pinot Blanc-Chardonnay'
 'Chardonnay-Pinot Blanc' 'Saperavi-Merlot' 'Malvasia Istriana' 'Torontel'
 'Picapoll' 'Zierfandler-Rotgipfler' 'Malvasia Fina' 'Chinuri' 'Muscatel'
 'Sousão' 'Silvaner-Traminer' 'Syrah-Carignan' 'Bukettraube' 'Muskat'
 'Argaman' 'Provence white blend' 'Touriga Franca' 'Morillon'
 'Carignan-Syrah' 'Aidani' 'Viognier-Grenache Blanc' 'Albarossa'
 'Sauvignon Blanc-Verdejo' 'Grenache-Mourvèdre' 'Tannat-Syrah'
 'Seyval Blanc' 'Tocai Rosso' 'Pinot-Chardonnay' 'Moscatel Graúdo'
 'Pigato' 'Siegerrebe' 'Bombino Bianco' 'Trebiano-Malvasia' 'Magliocco'
 'Verduzzo Friulano' 'Vespaiole' 'Marzemino' 'Tempranillo-Malbec'
 'Crespiello' 'Cabernet Franc-Tempranillo' 'Gouveio' 'Caprettone'
 'Garnacha-Graciano' 'Mataro' "Pineau d'Aunis" 'Bual' 'Sercial'
 'Moscato di Noto' 'Sauvignonasse' 'Madeira Blend' 'St. George' 'Rebula'
 'Pallagrello Bianco' 'Vilana' 'Pelaverga Piccolo'
 'Syrah-Grenache-Viognier' 'Alvarelhão' 'Durif' 'Angevine'
 'Semillon-Chardonnay' 'Pinot Blanc-Pinot Noir' 'Manzoni' 'Maréchal Foch'
 'Blauer Portugieser' 'Tocai' 'Shiraz-Malbec' 'Cabernet Moravia'
 'Espadeiro' 'País' 'Altesse' 'Avesso' 'Grignolino' 'Mandilaria' 'Freisa'
 'Merlot-Shiraz' 'Dafni' 'Xynisteri' 'Grechetto' 'Roschetto' 'Sideritis'
 'Pinotage-Merlot' 'Asprinio' 'Grolleau' 'Gagnano' 'Ansonica'
 'Sangiovese Cabernet' 'Tinta Barroca' 'Syrah-Bonarda' 'Marsanne-Viognier'
 'Azal' 'Durello' 'Syrah-Malbec' 'Malbec-Cabernet Franc' 'Franconia'
 'Rufete' 'Parraleta' 'St. Vincent' 'Groppello' 'Athiri'
 'Muscat of Alexandria' 'Malvoisie' 'Colorino' 'Merlot-Grenache'
 'Terret Blanc' 'Chardonel' 'Macabeo-Gewürztraminer' 'Grenache Gris'
 'Rabigato' 'Muscat Hamburg' 'Sarba' 'Irsai Oliver'
 'Chardonnay-Pinot Gris' 'Vermentino Nero' 'Pardina' 'Apple' 'Clairette'
 'Sauvignon Musqué' 'Shiraz-Merlot' 'Viognier-Valdiguié' 'Chardonelle'
 'Malmsey' 'Tinta Negra Mole' 'Pinot Grigio-Chardonnay' 'Muscadet'
 'Viura-Sauvignon Blanc' 'Huxelrebe' 'Tokay Pinot Gris'
 'Chardonnay-Pinot Grigio' 'Moristel' 'Carnelian']

1) Обработка пропусков в данных

1.1) Простые стратегии - удаление или заполнение нулями

```
df_copy = df.copy()
df_copy.head(10)
```

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
0	0	US	This tremendous 100% varietal wine hails from ...	Martha's Vineyard	96	235.0	California	Napa Valley	Napa	Cabernet Sauvignon	Heitz
1	1	Spain	Ripe aromas of fig, blackberry	Carodorum Selección Especial Reserva	96	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodríguez

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
			and cassis are ...								
2	2	US	Mac Watson honors the memory of a wine once ma...	Special Selected Late Harvest	96	90.0	California	Knights Valley	Sonoma	Sauvignon Blanc	Macaul
3	3	US	This spent 20 months in 30% new French oak, an...	Reserve	96	65.0	Oregon	Willamette Valley	Willamette Valley	Pinot Noir	Ponzi
4	4	France	This is the top wine from La Bégude, named aft...	La Brûlade	95	66.0	Provence	Bandol	NaN	Provence red blend	Domair de la Bégude
5	5	Spain	Deep, dense and pure from the opening bell, th...	Numanthia	95	73.0	Northern Spain	Toro	NaN	Tinta de Toro	Numan
6	6	Spain	Slightly gritty black-fruit aromas include a s...	San Román	95	65.0	Northern Spain	Toro	NaN	Tinta de Toro	Mauroc
7	7	Spain	Lush cedary black-fruit aromas are luxe and of...	Carodorum Único Crianza	95	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodríguez
8	8	US	This re-named vineyard was formerly bottled as...	Silice	95	65.0	Oregon	Chehalem Mountains	Willamette Valley	Pinot Noir	Bergstr
9	9	US	The producer sources from two blocks of the vi...	Gap's Crown Vineyard	95	60.0	California	Sonoma Coast	Sonoma	Pinot Noir	Blue Fa

```
# Удаление колонок, содержащих пустые значения
df_new_1 = df_copy.dropna(axis=1, how='any')
(df.shape, df_new_1.shape)
```

```
((150930, 11), (150930, 5))
```

```
# Удаление строк, содержащих пустые значения
df_new_2 = df_copy.dropna(axis=0, how='any')
(df.shape, df_new_2.shape)
```

```
((150930, 11), (39241, 11))
```

```
# Заполнение пустых значений (некорректно занулять значение категориального поля)
# Столбец region_2 может содержать пустые значения
```

```

categorical_cols = ['country', 'province', 'region_1', 'variety']
df_new_3 = df_copy.copy()
# Заполняем пропущенные значения в категориальных столбцах наиболее часто встречающимся значением (модой)
for col in categorical_cols:
    if col in df_new_3.columns:
        most_frequent_value = df_new_3[col].mode()[0]
        df_new_3[col] = df_new_3[col].fillna(most_frequent_value)
    else:
        print(f"Столбец '{col}' не найден в датасете.")

# Проверяем, что пропущенных значений больше нет в категориальных столбцах
print(df_new_3[categorical_cols].isnull().sum())

```

```

country      0
province     0
region_1     0
variety      0
dtype: int64

```

```

# Выводим первые несколько строк, чтобы убедиться, что все в порядке
df_new_3.head(10)

```

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
0	0	US	This tremendous 100% varietal wine hails from ...	Martha's Vineyard	96	235.0	California	Napa Valley	Napa	Cabernet Sauvignon	Heitz
1	1	Spain	Ripe aromas of fig, blackberry and cassis are ...	Carodorum Selección Especial Reserva	96	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodríguez
2	2	US	Mac Watson honors the memory of a wine once ma...	Special Selected Late Harvest	96	90.0	California	Knights Valley	Sonoma	Sauvignon Blanc	Macaul
3	3	US	This spent 20 months in 30% new French oak, an...	Reserve	96	65.0	Oregon	Willamette Valley	Willamette Valley	Pinot Noir	Ponzi
4	4	France	This is the top wine from La Bégude, named aft...	La Brûlade	95	66.0	Provence	Bandol	NaN	Provence red blend	Domaine de la Bégude
5	5	Spain	Deep, dense and pure from the opening bell, th...	Numanthia	95	73.0	Northern Spain	Toro	NaN	Tinta de Toro	Numan
6	6	Spain	Slightly gritty black-fruit aromas include a s...	San Román	95	65.0	Northern Spain	Toro	NaN	Tinta de Toro	Mauroc
7	7	Spain	Lush cedary black-fruit	Carodorum Único Crianza	95	110.0	Northern Spain	Toro	NaN	Tinta de Toro	Bodega Carmer Rodríguez

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
			aromas are luxe and of...								
8	8	US	This re-named vineyard was formerly bottled as...	Silice	95	65.0	Oregon	Chehalem Mountains	Willamette Valley	Pinot Noir	Bergstr
9	9	US	The producer sources from two blocks of the vi...	Gap's Crown Vineyard	95	60.0	California	Sonoma Coast	Sonoma	Pinot Noir	Blue Fa

1.2) "Внедрение значений" - импьютация (imputation)

Обработка пропусков в числовых данных

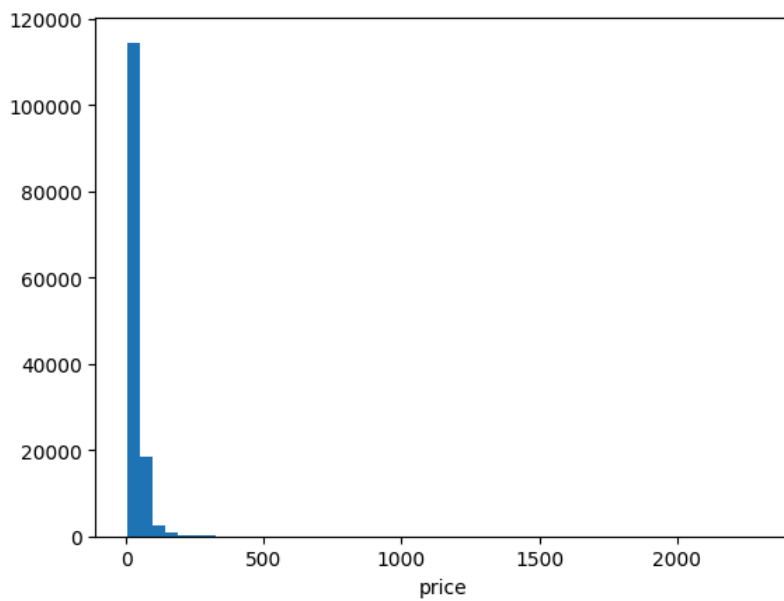
```
# Выберем числовые колонки с пропущенными значениями
# Цикл по колонкам датасета
num_cols = []
for col in df.columns:
    # Количество пустых значений
    temp_null_count = df[df[col].isnull()].shape[0]
    dt = str(df[col].dtype)
    if temp_null_count>0 and (dt=='float64' or dt=='int64'):
        num_cols.append(col)
        temp_perc = round((temp_null_count / total_count) * 100.0, 2)
        print('Колонка {}. Тип данных {}. Количество пустых значений {}, {}%'.format(col, dt, temp_null_count, temp_perc))
```

Колонка price. Тип данных float64. Количество пустых значений 13695, 9.07%.

```
# Фильтр по колонкам с пропущенными значениями
df_num = df[num_cols]
df_num.head(10)
```

	price
0	235.0
1	110.0
2	90.0
3	65.0
4	66.0
5	73.0
6	65.0
7	110.0
8	65.0
9	60.0

```
%matplotlib inline
# Гистограмма по признакам
for col in df_num:
    plt.hist(df[col], 50)
    plt.xlabel(col)
    plt.show()
```



```
data_num_price = df_num[['price']]
data_num_price.head(10)
```

	price
0	235.0
1	110.0
2	90.0
3	65.0
4	66.0
5	73.0
6	65.0
7	110.0
8	65.0
9	60.0

```
from sklearn.impute import SimpleImputer
from sklearn.impute import MissingIndicator

# Фильтр для проверки заполнения пустых значений
indicator = MissingIndicator()
mask_missing_values_only = indicator.fit_transform(data_num_price)
mask_missing_values_only
```

```
array([[False],
       [False],
       [False],
       ...,
       [False],
       [False],
       [False]])
```

```
# С помощью класса SimpleImputer можно проводить импутацию различными показателями центра распределения
strategies=['mean', 'median', 'most_frequent']
```

```
def test_num_impute(strategy_param):
    imp_num = SimpleImputer(strategy=strategy_param)
    data_num_imp = imp_num.fit_transform(data_num_price)
    return data_num_imp[mask_missing_values_only]
```

```
strategies[0], test_num_impute(strategies[0])
```

```
('mean',  
 array([33.13148249, 33.13148249, 33.13148249, ..., 33.13148249,  
        33.13148249, 33.13148249]))
```

```
strategies[1], test_num_impute(strategies[1])
```

```
('median', array([24., 24., 24., ..., 24., 24., 24.]))
```

```
strategies[2], test_num_impute(strategies[2])
```

```
('most_frequent', array([20., 20., 20., ..., 20., 20., 20.]))
```

```
# Более сложная функция, которая позволяет задавать колонку и вид импутации  
def test_num_impute_col(dataset, column, strategy_param):  
    temp_data = dataset[[column]]  
  
    indicator = MissingIndicator()  
    mask_missing_values_only = indicator.fit_transform(temp_data)  
  
    imp_num = SimpleImputer(strategy=strategy_param)  
    data_num_imp = imp_num.fit_transform(temp_data)  
  
    filled_data = data_num_imp[mask_missing_values_only]  
  
    return column, strategy_param, filled_data.size, filled_data[0], filled_data[filled_data.size-1]  
  
df[['price']].describe()
```

	price
count	137235.000000
mean	33.131482
std	36.322536
min	4.000000
25%	16.000000
50%	24.000000
75%	40.000000
max	2300.000000

```
test_num_impute_col(df, 'price', strategies[0])
```

```
('price',  
 'mean',  
 13695,  
 np.float64(33.13148249353299),  
 np.float64(33.13148249353299))
```

```
test_num_impute_col(df, 'price', strategies[1])
```

```
('price', 'median', 13695, np.float64(24.0), np.float64(24.0))
```

```
test_num_impute_col(df, 'price', strategies[2])
```

```
('price', 'most_frequent', 13695, np.float64(20.0), np.float64(20.0))
```

```
df_itog = df.copy()
imp_num = SimpleImputer(strategy='most_frequent')
df_itog = pd.DataFrame(imp_num.fit_transform(df_itog), columns=df_itog.columns)

df_itog.head(10)
```

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
0	0	US	This tremendous 100% varietal wine hails from ...	Martha's Vineyard	96	235.0	California	Napa Valley	Napa	Cabernet Sauvignon	Heitz
1	1	Spain	Ripe aromas of fig, blackberry and cassis are ...	Carodorum Selección Especial Reserva	96	110.0	Northern Spain	Toro	Central Coast	Tinta de Toro	Bodega Carmer Rodríguez
2	2	US	Mac Watson honors the memory of a wine once ma...	Special Selected Late Harvest	96	90.0	California	Knights Valley	Sonoma	Sauvignon Blanc	Macaul
3	3	US	This spent 20 months in 30% new French oak, an...	Reserve	96	65.0	Oregon	Willamette Valley	Willamette Valley	Pinot Noir	Ponzi
4	4	France	This is the top wine from La Bégude, named aft...	La Brûlade	95	66.0	Provence	Bandol	Central Coast	Provence red blend	Domaine de la Bégude
5	5	Spain	Deep, dense and pure from the opening bell, th...	Numanthia	95	73.0	Northern Spain	Toro	Central Coast	Tinta de Toro	Numan
6	6	Spain	Slightly gritty black-fruit aromas include a s...	San Román	95	65.0	Northern Spain	Toro	Central Coast	Tinta de Toro	Mauroc
7	7	Spain	Lush cedary black-fruit aromas are luxe and of...	Carodorum Único Crianza	95	110.0	Northern Spain	Toro	Central Coast	Tinta de Toro	Bodega Carmer Rodríguez
8	8	US	This re-named vineyard was formerly bottled as...	Silice	95	65.0	Oregon	Chehalem Mountains	Willamette Valley	Pinot Noir	Bergstr
9	9	US	The producer sources from two	Gap's Crown Vineyard	95	60.0	California	Sonoma Coast	Sonoma	Pinot Noir	Blue Fa

	Unnamed: 0	country	description	designation	points	price	province	region_1	region_2	variety	winery
			blocks of the vi...								

```
print(df_itog.isnull().sum())
```

```
Unnamed: 0      0
country         0
description      0
designation      0
points          0
price           0
province        0
region_1        0
region_2        0
variety         0
winery          0
dtype: int64
```

2) Кодирование категориальных признаков

2.1) Обработка пропусков в категориальных данных

```
# Выберем категориальные колонки с пропущенными значениями
# Цикл по колонкам датасета
cat_cols = []
for col in df.columns:
    # Количество пустых значений
    temp_null_count = df[df[col].isnull()].shape[0]
    dt = str(df[col].dtype)
    if temp_null_count > 0 and (dt == 'object'):
        cat_cols.append(col)
        temp_perc = round((temp_null_count / total_count) * 100.0, 2)
        print('Колонка {}. Тип данных {}. Количество пустых значений {}, {}%.'.format(col, dt, temp_null_count, temp_perc))
```

Колонка country. Тип данных object. Количество пустых значений 5, 0.0%.

Колонка designation. Тип данных object. Количество пустых значений 45735, 30.3%.

Колонка province. Тип данных object. Количество пустых значений 5, 0.0%.

Колонка region_1. Тип данных object. Количество пустых значений 25060, 16.6%.

Колонка region_2. Тип данных object. Количество пустых значений 89977, 59.62%.

```
data = df.copy()
temp_data = data[['country']]
temp_data.head(10)
```

	country
0	US
1	Spain
2	US
3	US
4	France
5	Spain
6	Spain
7	Spain
8	US
9	US


```
temp_data['country'].unique()
```

```
array(['US', 'Spain', 'France', 'Italy', 'New Zealand', 'Bulgaria',  
      'Argentina', 'Australia', 'Portugal', 'Israel', 'South Africa',  
      'Greece', 'Chile', 'Morocco', 'Romania', 'Germany', 'Canada',  
      'Moldova', 'Hungary', 'Austria', 'Croatia', 'Slovenia', nan,  
      'India', 'Turkey', 'Macedonia', 'Lebanon', 'Serbia', 'Uruguay',  
      'Switzerland', 'Albania', 'Bosnia and Herzegovina', 'Brazil',  
      'Cyprus', 'Lithuania', 'Japan', 'China', 'South Korea', 'Ukraine',  
      'England', 'Mexico', 'Georgia', 'Montenegro', 'Luxembourg',  
      'Slovakia', 'Czech Republic', 'Egypt', 'Tunisia', 'US-France'],  
      dtype=object)
```

```
print(temp_data['country'].isnull().sum())
```

5

```
# Импутация наиболее частыми значениями  
imp2 = SimpleImputer(missing_values=np.nan, strategy='most_frequent')  
data_imp2 = imp2.fit_transform(temp_data)  
data_imp2 = pd.DataFrame(data_imp2, columns = temp_data.columns)  
print(data_imp2['country'].isnull().sum())
```

0

```
# Импутация константой  
imp3 = SimpleImputer(missing_values=np.nan, strategy='constant', fill_value='NA')  
data_imp3 = imp3.fit_transform(temp_data)  
data_imp3 = pd.DataFrame(data_imp3, columns = temp_data.columns)  
print(data_imp3['country'].isnull().sum())
```

0

```
data_imp3['country'].unique()
```

```
array(['US', 'Spain', 'France', 'Italy', 'New Zealand', 'Bulgaria',  
      'Argentina', 'Australia', 'Portugal', 'Israel', 'South Africa',  
      'Greece', 'Chile', 'Morocco', 'Romania', 'Germany', 'Canada',  
      'Moldova', 'Hungary', 'Austria', 'Croatia', 'Slovenia', 'NA',  
      'India', 'Turkey', 'Macedonia', 'Lebanon', 'Serbia', 'Uruguay',  
      'Switzerland', 'Albania', 'Bosnia and Herzegovina', 'Brazil',  
      'Cyprus', 'Lithuania', 'Japan', 'China', 'South Korea', 'Ukraine',  
      'England', 'Mexico', 'Georgia', 'Montenegro', 'Luxembourg',  
      'Slovakia', 'Czech Republic', 'Egypt', 'Tunisia', 'US-France'],  
      dtype=object)
```

```
data_imp3[data_imp3['country'] == 'NA'].size
```

5

2.2) Преобразование категориальных признаков в числовые

```
data_enc = pd.DataFrame({'c1':data_imp2['country']})  
data_enc
```

	c1
0	US
1	Spain
2	US
3	US
4	France
...	...
150925	Italy
150926	France
150927	Italy
150928	France
150929	Italy

150930 rows × 1 columns

Кодирование категорий целочисленными значениями (label encoding)

Использование LabelEncoder

```
from sklearn.preprocessing import LabelEncoder
data_enc['c1'].unique()
```

```
array(['US', 'Spain', 'France', 'Italy', 'New Zealand', 'Bulgaria',
       'Argentina', 'Australia', 'Portugal', 'Israel', 'South Africa',
       'Greece', 'Chile', 'Morocco', 'Romania', 'Germany', 'Canada',
       'Moldova', 'Hungary', 'Austria', 'Croatia', 'Slovenia', 'India',
       'Turkey', 'Macedonia', 'Lebanon', 'Serbia', 'Uruguay',
       'Switzerland', 'Albania', 'Bosnia and Herzegovina', 'Brazil',
       'Cyprus', 'Lithuania', 'Japan', 'China', 'South Korea', 'Ukraine',
       'England', 'Mexico', 'Georgia', 'Montenegro', 'Luxembourg',
       'Slovakia', 'Czech Republic', 'Egypt', 'Tunisia', 'US-France'],
      dtype=object)
```

```
le = LabelEncoder()
data_enc_le = le.fit_transform(data_enc['c1'])
```

```
# Наименования категорий в соответствии с порядковыми номерами

# Свойство называется classes, потому что предполагается что мы решаем
# задачу классификации и каждое значение категории соответствует
# какому-либо классу целевого признака

le.classes_
```

```
array(['Albania', 'Argentina', 'Australia', 'Austria',
       'Bosnia and Herzegovina', 'Brazil', 'Bulgaria', 'Canada', 'Chile',
       'China', 'Croatia', 'Cyprus', 'Czech Republic', 'Egypt', 'England',
       'France', 'Georgia', 'Germany', 'Greece', 'Hungary', 'India',
       'Israel', 'Italy', 'Japan', 'Lebanon', 'Lithuania', 'Luxembourg',
       'Macedonia', 'Mexico', 'Moldova', 'Montenegro', 'Morocco',
       'New Zealand', 'Portugal', 'Romania', 'Serbia', 'Slovakia',
       'Slovenia', 'South Africa', 'South Korea', 'Spain', 'Switzerland',
       'Tunisia', 'Turkey', 'US', 'US-France', 'Ukraine', 'Uruguay'],
      dtype=object)
```

```
data_enc_le
```

```
array([44, 40, 44, ..., 22, 15, 22])
```

```
np.unique(data_enc_le)
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47])
```

Использование OrdinalEncoder

```
from sklearn.preprocessing import OrdinalEncoder
data_oe = data[['country']]
data_oe.head()
```

	country
0	US
1	Spain
2	US
3	US
4	France

```
imp4 = SimpleImputer(missing_values=np.nan, strategy='constant', fill_value='NA')
data_oe_filled = imp4.fit_transform(data_oe)
data_oe_filled
```

```
array(['US',
       'Spain',
       'US',
       ...,
       'Italy',
       'France',
       'Italy'], dtype=object)
```

```
oe = OrdinalEncoder()
data_enc_oe = oe.fit_transform(data_oe_filled)
data_enc_oe
```

```
array([[45.],
       [41.],
       [45.],
       ...,
       [22.],
       [15.],
       [22.]])
```

```
# Уникальные значения 1 признака
np.unique(data_enc_oe[:, 0])
```

```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12.,
        13., 14., 15., 16., 17., 18., 19., 20., 21., 22., 23., 24., 25.,
        26., 27., 28., 29., 30., 31., 32., 33., 34., 35., 36., 37., 38.,
        39., 40., 41., 42., 43., 44., 45., 46., 47., 48.]
```

```
# Наименования категорий в соответствии с порядковыми номерами
oe.categories_
```

```
[array(['Albania', 'Argentina', 'Australia', 'Austria',  
      'Bosnia and Herzegovina', 'Brazil', 'Bulgaria', 'Canada', 'Chile',  
      'China', 'Croatia', 'Cyprus', 'Czech Republic', 'Egypt', 'England',  
      'France', 'Georgia', 'Germany', 'Greece', 'Hungary', 'India',  
      'Israel', 'Italy', 'Japan', 'Lebanon', 'Lithuania', 'Luxembourg',  
      'Macedonia', 'Mexico', 'Moldova', 'Montenegro', 'Morocco', 'NA',  
      'New Zealand', 'Portugal', 'Romania', 'Serbia', 'Slovakia',  
      'Slovenia', 'South Africa', 'South Korea', 'Spain', 'Switzerland',  
      'Tunisia', 'Turkey', 'US', 'US-France', 'Ukraine', 'Uruguay'],  
      dtype=object)]
```

```
# Обратное преобразование  
oe.inverse_transform(data_enc_oe)
```

```
array(['US'],  
      ['Spain'],  
      ['US'],  
      ...,  
      ['Italy'],  
      ['France'],  
      ['Italy']], dtype=object)
```

Проблемы использования LabelEncoder и OrdinalEncoder

Необходимо отметить, что LabelEncoder и OrdinalEncoder могут использоваться только для категориальных признаков в номинальных шкалах

(для которых отсутствует порядок), например города, страны, названия рек и т.д.

Это связано с тем, что задать какой-либо порядок при кодировании с помощью LabelEncoder и OrdinalEncoder невозможно, они сортируют категории в лексикографическом порядке.

При этом кодирование целыми числами создает фиктивное отношение порядка ($1 < 2 < 3 < \dots$) которого не было в исходных номинальных шкалах.

Данное отношение порядка может негативно повлиять на построение модели машинного обучения.

Плюсы и минусы использования LabelEncoder и OrdinalEncoder:

LabelEncoder:

1. Вход: Один столбец
2. Назначение: В основном целевая переменная
3. Обработка: Автоматическое присвоение номеров
4. Множественные столбцы: Требуется обработка каждого столбца отдельно

OrdinalEncoder:

1. Матрица (DataFrame или NumPy array)
2. Назначение: В основном признаки (нецелевые)
3. Обработка: Учет порядка (если задан)
4. Множественные выборы: Может обрабатывать сразу несколько столбцов

Пример, когда OrdinalEncoder лучше, чем LabelEncoder. Используем случай, когда важен порядок.

Возьмем пример с размерами одежды и странами. Если мы хотим взять упорядочить страны, то нам не важен их порядок, поэтому любым странам может соответствовать любые индексы. В таком случае воспользуемся LabelEncoder.

В случае с размерами одежды лучше использовать OrdinalEncoder, так как размеры это не просто названия, но они еще и упорядочены. То есть XS->S->M->L->XL

```
# Пример для размеров одежды  
import pandas as pd  
from sklearn.preprocessing import OrdinalEncoder
```

```
data_clothes = {'Размер': ['M', 'XS', 'L', 'S', 'XL', 'M']}
df_clothes = pd.DataFrame(data_clothes)

mapping = [['XS', 'S', 'M', 'L', 'XL']]

encoder = OrdinalEncoder(categories=mapping)
df_clothes['КодРазмера'] = encoder.fit_transform(df_clothes[['Размер']])

print(df_clothes)
```

	Размер	КодРазмера
0	M	2.0
1	XS	0.0
2	L	3.0
3	S	1.0
4	XL	4.0
5	M	2.0

```
# Пример для стран
np.unique(data_enc_oe[:, 0])
```

```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12.,
        13., 14., 15., 16., 17., 18., 19., 20., 21., 22., 23., 24., 25.,
        26., 27., 28., 29., 30., 31., 32., 33., 34., 35., 36., 37., 38.,
        39., 40., 41., 42., 43., 44., 45., 46., 47., 48.])
```

```
# Наименования категорий в соответствии с порядковыми номерами
oe.categories_
```

```
[array(['Albania', 'Argentina', 'Australia', 'Austria',
        'Bosnia and Herzegovina', 'Brazil', 'Bulgaria', 'Canada', 'Chile',
        'China', 'Croatia', 'Cyprus', 'Czech Republic', 'Egypt', 'England',
        'France', 'Georgia', 'Germany', 'Greece', 'Hungary', 'India',
        'Israel', 'Italy', 'Japan', 'Lebanon', 'Lithuania', 'Luxembourg',
        'Macedonia', 'Mexico', 'Moldova', 'Montenegro', 'Morocco', 'NA',
        'New Zealand', 'Portugal', 'Romania', 'Serbia', 'Slovakia',
        'Slovenia', 'South Africa', 'South Korea', 'Spain', 'Switzerland',
        'Tunisia', 'Turkey', 'US', 'US-France', 'Ukraine', 'Uruguay'],
        dtype=object)]
```

3) Масштабирование данных

Термины "масштабирование" и "нормализация" часто используются как синонимы, но это неверно. Масштабирование предполагает изменение диапазона измерения величины, а нормализация - изменение распределения этой величины. В этом разделе рассматривается только масштабирование.

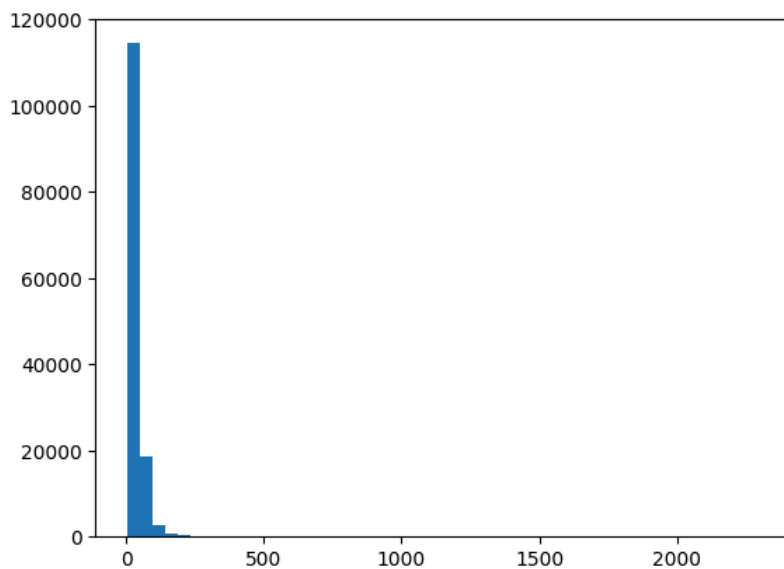
Если признаки лежат в различных диапазонах, то необходимо их нормализовать. Как правило, применяют два подхода:

1. MinMax масштабирование (В этом случае значения лежат в диапазоне от 0 до 1)
2. Масштабирование данных на основе Z-оценки (В этом случае большинство значений попадает в диапазон от -3 до 3)

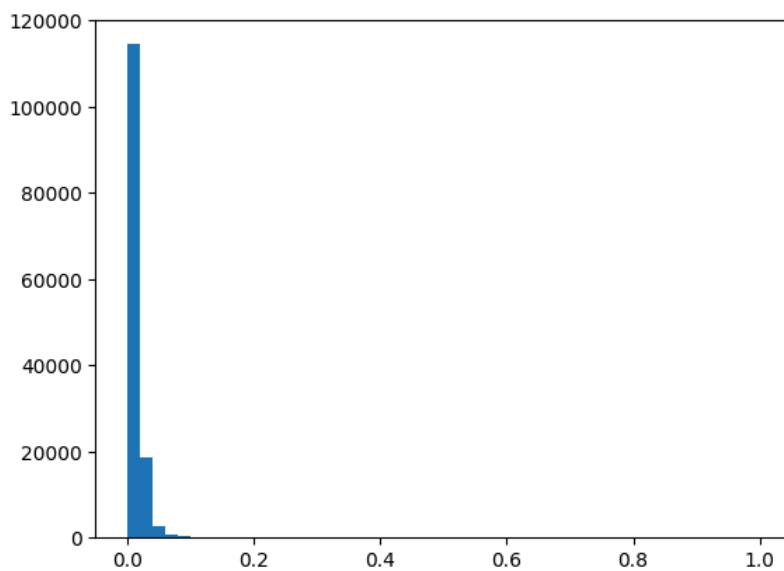
```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer
```

3.1) MinMax масштабирование

```
sc1 = MinMaxScaler()
sc1_data = sc1.fit_transform(data[['price']])
plt.hist(data['price'], 50)
plt.show()
```

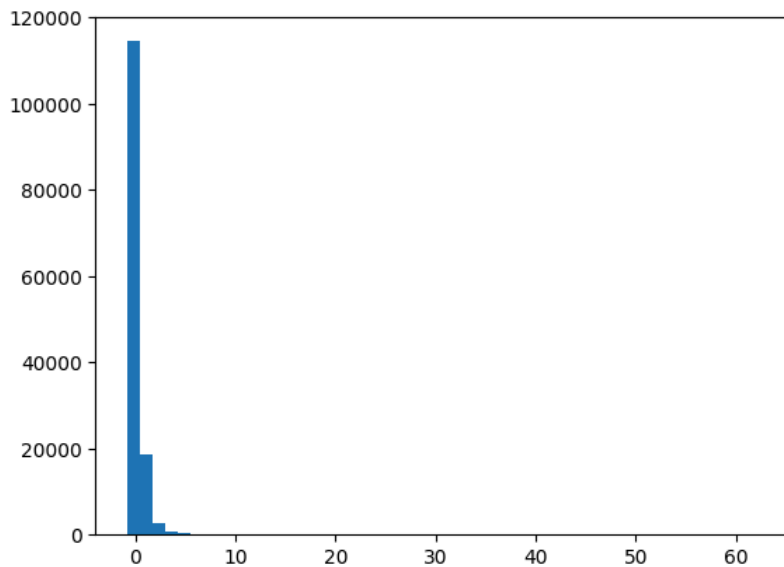


```
# В этом случае значения лежат в диапазоне от 0 до 1  
plt.hist(sc1_data, 50)  
plt.show()
```



3.2) Масштабирование данных на основе Z-оценки

```
# В этом случае большинство значений попадает в диапазон от -3 до 3  
sc2 = StandardScaler()  
sc2_data = sc2.fit_transform(data[['price']])  
plt.hist(sc2_data, 50)  
plt.show()
```



Масштабирование данных - это преобразование числовых признаков, чтобы привести их к определенному диапазону или распределению. Это нужно, чтобы признаки с разными единицами измерения или масштабами не вносили непропорциональный вклад в модели машинного обучения.

Виды масштабирования:

1. **MinMax Scaling**: Приводит значения признаков к диапазону от 0 до 1. Работает, вычитая минимум и деля на разницу между максимумом и минимумом. Подходит, когда важно сохранить отношения между данными и когда нет выбросов.
2. **Z-score Standardization** (Standard Scaling): Преобразует значения, чтобы они имели среднее значение 0 и стандартное отклонение 1. Работает, вычитая среднее и деля на стандартное отклонение. Хорошо, когда важна форма распределения (например, для линейных моделей) и когда есть выбросы. Большинство значений оказываются в диапазоне от -3 до 3, но это не гарантируется.